

Simplificação do Sistema de Autenticação

Resumo

O sistema de autenticação foi simplificado para uso local. A autenticação agora é baseada apenas em **nickname**, sem necessidade de senha, tokens JWT ou OAuth.

O que foi removido

1. OAuth Google

- Removida toda a lógica de autenticação via Google OAuth2
- Removidas as rotas `/auth/google` e `/auth/google/callback`
- Essas rotas agora retornam erro 410 (Gone) para compatibilidade

2. Sessões e Passport.js

- Removida a configuração de `express-session`
- Removida a inicialização do `passport.js`
- O middleware `sessionConfig` e `passport` ainda existem como stubs para compatibilidade

3. Tokens e Verificação de Autenticação

- Não há mais geração de tokens JWT
- Não há mais verificação de sessão ativa
- O middleware `isAuthenticated` agora sempre permite acesso

4. Campos removidos do User (conceitual)

- `googleId`, `accessToken`, `refreshToken` não são mais utilizados pelo sistema de auth
- O schema Prisma ainda pode conter esses campos para compatibilidade com dados existentes

Como funciona agora

Login Simplificado

Endpoint: `POST /api/auth/login`

Request:

```
{
  "nickname": "seu_nickname"
}
```

Response (sucesso):

```
{
  "success": true,
  "message": "Login realizado com sucesso",
  "user": {
    "id": "uuid",
    "name": "Nome do Usuário",
    "nickname": "seu_nickname",
    "email": "email@exemplo.com",
    "avatarUrl": null,
    "role": "Admin"
  }
}
```

Response (erro):

```
{
  "success": false,
  "error": "Not Found",
  "message": "Usuário não encontrado com este nickname"
}
```

Identificando o Usuário nas Requisições

O sistema aceita o usuário de várias formas:

1. Header HTTP:

```
X-User-Nickname: seu_nickname
```

2. Body da requisição:

```
json
{
  "nickname": "seu_nickname",
  // ou
  "userNickname": "seu_nickname",
  // ou
  "createdByNickname": "seu_nickname",
  // ou
  "createdById": "uuid-do-usuario"
}
```

3. Query Parameter:

```
?nickname=seu_nickname
```

Exemplos de Uso

Login

```
curl -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"nickname": "alexandre}"'
```

Verificar Usuário Atual

```
curl http://localhost:3001/api/auth/me \
-H "X-User-Nickname: alexandre"
```

Criar Tarefa

```
curl -X POST http://localhost:3001/api/tasks \
-H "Content-Type: application/json" \
-H "X-User-Nickname: alexandre" \
-d '{
  "title": "Nova Tarefa",
  "description": "Descrição da tarefa",
  "projectId": "uuid-projeto",
  "statusId": "uuid-status",
  "priorityId": "uuid-prioridade"
}'
```

Ou passando o usuário no body:

```
curl -X POST http://localhost:3001/api/tasks \
-H "Content-Type: application/json" \
-d '{
  "title": "Nova Tarefa",
  "description": "Descrição da tarefa",
  "projectId": "uuid-projeto",
  "statusId": "uuid-status",
  "priorityId": "uuid-prioridade",
  "nickname": "alexandre"
}'
```

Finalizar Tarefa

```
curl -X PATCH http://localhost:3001/api/tasks/{taskId}/finalize \
-H "Content-Type: application/json" \
-d '{
  "nickname": "alexandre",
  "executionNotes": "Tarefa concluída com sucesso"
}'
```

Ou usando userId:

```
curl -X PATCH http://localhost:3001/api/tasks/{taskId}/finalize \
-H "Content-Type: application/json" \
-d '{
  "userId": "uuid-do-usuario",
  "executionNotes": "Tarefa concluída com sucesso"
}'
```

Verificar Status de Autenticação

```
# Por nickname
curl -X POST http://localhost:3001/api/auth/check \
-H "Content-Type: application/json" \
-d '{"nickname": "alexandre"}'

# Por userId
curl -X POST http://localhost:3001/api/auth/check \
-H "Content-Type: application/json" \
-d '{"userId": "uuid-do-usuario"}'
```

Registrar Novo Usuário

```
curl -X POST http://localhost:3001/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "name": "Nome Completo",
  "nickname": "meu_nickname",
  "email": "email@exemplo.com"
}'
```

Mudanças nas Rotas

Rota	Método	Status	Descrição
/api/auth/login	POST	✓ Ativo	Login por nickname
/api/auth/me	GET	✓ Ativo	Obter usuário atual (requer header ou query)
/api/auth/check	GET/POST	✓ Ativo	Verificar autenticação
/api/auth/logout	POST	✓ Ativo	Logout (apenas retorna sucesso)
/api/auth/register	POST	✓ Ativo	Registrar novo usuário
/api/auth/google	GET	✗ Removido	Retorna erro 410
/api/auth/google/callback	GET	✗ Removido	Retorna erro 410

Arquivos Modificados

1. `src/middlewares/authMiddleware.js` - Simplificado para extração de usuário
2. `src/controllers/authController.js` - Removido OAuth, simplificado login
3. `src/routes/authRoutes.js` - Removidas rotas OAuth

4. `src/controllers/taskController.js` - Aceita nickname no body/header
5. `src/services/userService.js` - Adicionado campo nickname nos selects
6. `src/server.js` - Removido passport/session, adicionado middleware extractUser

Considerações de Segurança

⚠ Este sistema foi simplificado para uso local pessoal.

- Não há verificação de senha
- Não há tokens de autenticação
- Qualquer pessoa com acesso à rede pode fazer login como qualquer usuário
- **Não use em produção pública sem adicionar camadas de segurança**

Compatibilidade com Frontend

O frontend deve:

1. Fazer login com `POST /api/auth/login` passando apenas `nickname`
2. Armazenar o objeto `user` retornado (id, nickname, name, etc.)
3. Enviar o `nickname` ou `userId` nas requisições subsequentes via:
 - Header `X-User-Nickname`
 - Ou no body da requisição

Documentação gerada em: 2026-03-10